

LEAD University

ULEAD SJ - IIC2026 - G1 - Programación Web - TTCT0012 - 888

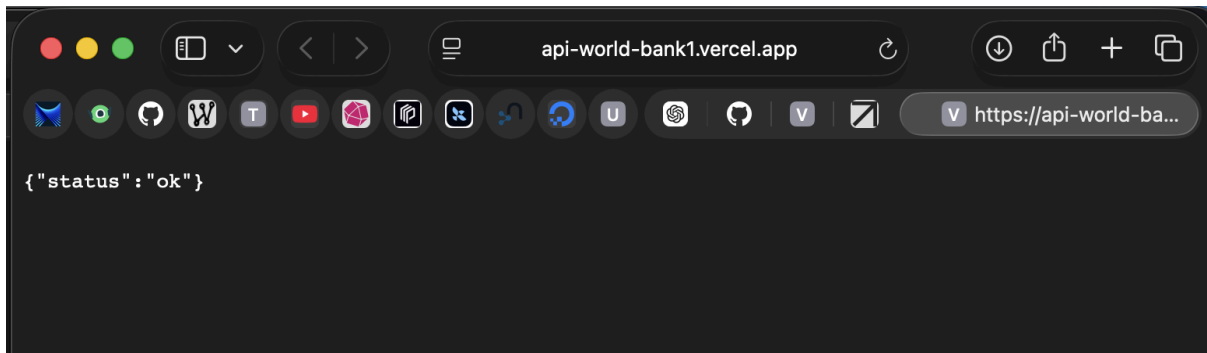
Profesor: Alejandro Zamora

Entregable: Trabajo Grupal 2

Estudiantes: Aron Vargas, Adrián González, Ana María Ramírez y Kristhel Porras

Recursos

Evidencia conexión API



Link Vercel: <https://api-world-bank1.vercel.app/>

Link repo: https://github.com/anamaria29/API_WorldBank

Explicación buenas prácticas

Separación frontend/backend

Por qué: Permite que cada capa evolucione de forma independiente (cambiar el diseño sin tocar la lógica de negocio, o cambiar de proveedor de datos sin tocar el frontend) y reduce el acoplamiento. También es más seguro: el frontend nunca habla directamente con la World Bank API, solo con tu propio backend.

Módulo independiente para consumir la API externa (world_bank.py)

Por qué: Si mañana cambia la URL, el formato de respuesta, o se reemplaza la World Bank API por otra fuente, solo se modifica ese archivo. Aísla el "punto de fallo" externo del resto de la aplicación (principio de responsabilidad única).

Manejo de errores tipados + validación de respuestas

Por qué: Las APIs externas pueden fallar, tardar o devolver datos inesperados. Sin manejo de errores, un fallo externo tumbaría toda la app o expondría trazas internas al usuario. Aquí se captura el error, se registra en logs (para debug) y se muestra un mensaje seguro al usuario (no información técnica sensible).

Timeout en las solicitudes

Por qué: Evita que una solicitud a la World Bank API se quede colgada indefinidamente si el servicio externo no responde, lo que dejaría el backend bloqueado (mala experiencia y riesgo de denegación de servicio interna).

Validación de códigos ISO de país

Por qué: Es una validación en el borde del sistema (input del usuario). Evita solicitudes malformadas a la API externa y previene que un usuario envíe datos arbitrarios que no correspondan a lo esperado.

Inyección de dependencias mediante un gateway

Por qué: Desacopla CountryComparisonService de la implementación concreta de WorldBankClient. Esto facilita testear el servicio con un mock del gateway, sin depender de la API real (pruebas más rápidas y confiables).

Configuración mediante variables de entorno

Por qué: Evita hardcodear URLs o timeouts en el código. Permite usar distintos valores en desarrollo, pruebas y producción sin modificar el código fuente, y evita filtrar configuración sensible en el repositorio.

Pruebas unitarias, linting y análisis de seguridad en CI

Por qué: Automatiza la detección de errores, mal estilo de código y vulnerabilidades conocidas (Bandit, pip-audit) antes de que el código llegue a producción. Esto reduce el riesgo de introducir bugs o dependencias vulnerables en cada cambio.

textContent en vez de insertar HTML externo

Por qué: Previene ataques XSS. Si algún día la data mostrada viniera manipulada, textContent no interpreta HTML/JS, así que no se ejecutaría código malicioso en el navegador del usuario.

Contenedor sin usuario root, filesystem de solo lectura, capacidades restringidas

Por qué: Aplica el principio de menor privilegio. Si un atacante comprometiera el contenedor, tendría permisos mínimos: no podría escribir en el sistema de archivos ni escalar privilegios fácilmente.

No usar Supabase

Por qué: La aplicación no persiste datos (no hay usuarios, historiales, etc.), así que agregar una base de datos sería una dependencia innecesaria que aumenta la superficie de ataque sin aportar valor funcional. Es una decisión consciente de mantener la solución simple (principio YAGNI).